

Python — La visualisation de données

Document 50 — PyQtGraph

Série « Visualisation » · temps réel et haute performance

Table des matières

1. Pourquoi PyQtGraph : la visualisation temps réel

2. Première image : une fenêtre de tracé

2.1 La boucle d'événements Qt

2.2 PlotWidget et la courbe

3. Un graphique complet

4. Les types de tracés

5. Le cœur du sujet : l'animation temps réel

5.1 setData : mettre à jour sans tout redessiner

5.2 QTimer : la boucle de rafraîchissement

6. Plusieurs graphiques et interactivité

7. Performance : les bons réflexes

8. PyQtGraph ou Matplotlib animation ?

Récapitulatif

Questions de vérification

1. Pourquoi PyQtGraph : la visualisation temps réel

Les bibliothèques vues jusqu'ici excellent sur des données figées. Dès qu'il faut rafraîchir un graphique des dizaines de fois par seconde — un flux de prix en direct, un capteur, un carnet d'ordres — elles peinent : Matplotlib redessine tout à chaque image, ce qui sature vite. PyQtGraph est conçu pour ce cas précis : le temps réel, à haute fréquence et basse latence.

La différence tient à la nature de la sortie : PyQtGraph n'est pas une image ni une page web, mais une *fenêtre d'application de bureau*, bâtie sur Qt et accélérée par le GPU. C'est l'étape 4 du pipeline (document 00) dans sa version « flux en direct ».

Atout	Ce que ça change
Temps réel	Des milliers de points rafraîchis sans saturer l'affichage.
Basse latence	Rendu accéléré (OpenGL) : la courbe suit le flux.
Fenêtre Qt	Une vraie application de bureau, interactive (zoom, pan fluides).
Léger	Conçu pour tourner en continu, sans fuite de mémoire.

Pourquoi ça compte. Le critère n'est pas l'esthétique mais la cadence. Si le graphique se met à jour plusieurs fois par seconde en continu, PyQtGraph ; s'il est tracé une fois puis exporté, Matplotlib. On choisit selon la vitesse de rafraîchissement, pas selon le rendu.

2. Première image : une fenêtre de tracé

2.1 La boucle d'événements Qt

PyQtGraph repose sur Qt, une bibliothèque d'interfaces. Tout programme Qt suit le même squelette : créer une application, ouvrir une fenêtre, puis lancer la boucle d'événements qui maintient la fenêtre vivante et réactive.

```
import pyqtgraph as pg
from pyqtgraph.Qt import QtWidgets
```

```
app = QtWidgets.QApplication([])      # 1. l'application Qt
win = pg.PlotWidget(title="Prix en direct")
win.show()                            # 2. ouvrir la fenetre
app.exec()                            # 3. la boucle d'evenements (bloquante)
```

`app.exec()` est bloquant : le programme y reste tant que la fenêtre est ouverte. C'est dans cette boucle que les mises à jour temps réel viendront s'insérer (section 5).

2.2 PlotWidget et la courbe

PlotWidget est la zone de tracé — l'équivalent d'un Axes Matplotlib. On y crée une courbe avec `plot()`, et l'on garde la référence à cette courbe pour la mettre à jour ensuite.

```
courbe = win.plot(xs, ys, pen="y") # garde la reference 'courbe'
```

Pourquoi ça compte. Retenir le squelette `QApplication -> fenêtre -> exec()`, et surtout garder la référence à l'objet courbe renvoyé par `plot()`. C'est lui qu'on actualisera, sans jamais recréer le graphique.

3. Un graphique complet

On habille la fenêtre — titre, axes, légende, grille — par des méthodes du PlotWidget, proches de l'esprit Matplotlib mais propres à PyQtGraph.

```
win = pg.PlotWidget()
win.setTitle("ES - flux en direct")
win.setLabel("left", "Prix", units="USD")
win.setLabel("bottom", "Temps", units="s")
win.showGrid(x=True, y=True, alpha=0.3)
win.addLegend()
```

```
courbe = win.plot(xs, ys, pen=pg.mkPen("c", width=2), name="ES")
```

Méthode	Rôle
<code>setTitle(...)</code>	Titre de la fenêtre de tracé.
<code>setLabel(cote, ...)</code>	Libellé d'un axe (left, bottom...), avec unités.
<code>showGrid(x, y)</code>	Affiche la grille.
<code>addLegend()</code>	Légende, à partir des <code>name=</code> des courbes.
<code>setYRange / setXRange</code>	Fixe les bornes d'un axe (utile en temps réel).

Le pen, pas la couleur. Dans PyQtGraph, l'apparence d'une ligne se règle par un « pen » (stylo) : `pg.mkPen(couleur, width=...)`. C'est l'analogue de `color + linewidth`, regroupés en un objet.

4. Les types de tracés

PyQtGraph couvre les tracés courants, toujours dans l'optique performance. La plupart passent par `plot()` ; certains ont leur classe dédiée.

Élément	Type	Quand l'utiliser
<code>plot(x, y)</code>	Ligne / points	Série temporelle, flux de prix.
<code>ScatterPlotItem</code>	Nuage de points	Beaucoup de points indépendants.
<code>BarGraphItem</code>	Barres	Volumes, histogrammes de marché.
<code>ImageItem</code>	Image / carte	Heatmap, carnet d'ordres, spectrogramme.
<code>CandlestickItem</code>	Chandelier	OHLC boursier (élément personnalisé).

Pour un nuage très dense rafraîchi en continu, `ScatterPlotItem` avec `pxMode=True` dessine des points de taille fixe en pixels, beaucoup plus vite.

Pourquoi ça compte. Le type change, mais le principe de performance reste : on crée l'élément une fois, on le garde, et on n'actualise que ses données. Recréer un élément à chaque image est l'erreur qui ruine la fluidité.

5. Le cœur du sujet : l'animation temps réel

5.1 setData : mettre à jour sans tout redessiner

C'est la clé de la performance. Plutôt que de retracer la courbe, on remplace ses données avec `setData()`. PyQtGraph ne redessine que ce qui change.

```
courbe = win.plot(xs, ys)

# plus tard, a chaque nouveau point :
xs.append(t)
ys.append(prix)
courbe.setData(xs, ys)           # mise a jour, pas de recreation
```

5.2 QTimer : la boucle de rafraîchissement

Pour rafraîchir à intervalle régulier, on n'écrit pas de boucle while (elle bloquerait l'interface). On confie le rythme à un `QTimer`, qui appelle une fonction de mise à jour toutes les N millisecondes, sans figer la fenêtre.

```
from pyqtgraph.Qt import QtCore

def update():
    t, prix = lire_prochain_prix()    # source du flux
    xs.append(t); ys.append(prix)
    courbe.setData(xs, ys)

timer = QtCore.QTimer()
timer.timeout.connect(update)        # appeler update a chaque tic
timer.start(50)                      # toutes les 50 ms (20 images/s)
app.exec()
```

Pourquoi ça compte. Le duo gagnant : `setData` (mettre à jour les données, pas recréer le graphique) + `QTimer` (cadencer sans bloquer l'interface). C'est ce schéma, et non la puissance brute, qui rend l'affichage temps réel fluide.

6. Plusieurs graphiques et interactivité

Pour empiler plusieurs tracés — prix, volume, indicateurs — on utilise un `GraphicsLayoutWidget`, qui dispose les `PlotItem` en grille. Chaque sous-graphique se rafraîchit indépendamment.

```
win = pg.GraphicsLayoutWidget()
p_prix = win.addPlot(row=0, col=0, title="Prix")
p_vol = win.addPlot(row=1, col=0, title="Volume")
p_vol.setXLink(p_prix)          # axes x synchronisés

c_prix = p_prix.plot(pen="c")
b_vol = pg.BarGraphItem(x=[], height=[], width=0.6)
p_vol.addItem(b_vol)
```

Outil	Rôle
<code>GraphicsLayoutWidget</code>	Conteneur en grille de plusieurs <code>PlotItem</code> .
<code>addPlot(row, col)</code>	Ajoute un sous-graphique à une position.
<code>setXLink(autre)</code>	Synchronise l'axe x (zoom/pan communs).
<code>InfiniteLine</code>	Curseur / seuil mobile (prix d'alerte).
<code>LinearRegionItem</code>	Zone sélectionnable (plage temporelle).

Une interactivité native. Le zoom à la molette et le pan au glissé sont actifs d'office, et restent fluides même pendant le rafraîchissement — un avantage décisif sur Matplotlib pour le suivi en direct.

7. Performance : les bons réflexes

PyQtGraph est rapide, mais quelques réflexes font la différence entre 5 et 200 images par seconde. Ils découlent tous du même principe : éviter de recalculer ce qui n'a pas changé.

Réflexe	Pourquoi
<code>setData</code> , jamais recréer	Recréer un objet par image effondre la cadence.
Borner les données	Garder une fenêtre glissante (N derniers points), pas tout l'historique.
Fixer les axes	<code>setYRange</code> évite le recalcul d'échelle à chaque image.
<code>useOpenGL(True)</code>	Active l'accélération GPU pour les courbes denses.
Grouper les mises à jour	Un seul <code>setData</code> par image, pas un par point.

```
pg.setConfigOptions(antialias=True, useOpenGL=True)
```

```
# fenetre glissante : ne garder que les 1000 derniers points
MAX = 1000
if len(xs) > MAX:
    xs = xs[-MAX:]; ys = ys[-MAX:]
```

Pourquoi ça compte. La performance temps réel n'est pas qu'une affaire de bibliothèque : elle vient surtout de la discipline du code — fenêtre glissante, axes fixes, une seule mise à jour par image. PyQtGraph donne l'outil ; ces réflexes en tirent la vitesse.

8. PyQtGraph ou Matplotlib animation ?

Matplotlib sait aussi animer, via FuncAnimation. La question n'est donc pas « lequel peut le faire » mais « lequel pour quelle cadence ».

Critère	Lequel choisir
Cadence élevée	PyQtGraph : conçu pour des dizaines d'images/s soutenues.
Animation simple	Matplotlib FuncAnimation : suffisant à basse fréquence.
Application de bureau	PyQtGraph : fenêtre Qt interactive et durable.
Export GIF/MP4	Matplotlib animation : sait écrire un fichier vidéo.
Écosystème connu	Matplotlib : API déjà maîtrisée (document 10).

La règle simple. Pour une animation courte qu'on exporte en GIF, ou un rafraîchissement lent, Matplotlib FuncAnimation suffit et reste familier. Pour un flux de marché en direct, soutenu et interactif, PyQtGraph est le bon outil.

Pourquoi ça compte. Les deux ne s'opposent pas sur tout : ils visent des cadences différentes. Matplotlib animation pour le simple et l'exportable ; PyQtGraph pour le temps réel exigeant. Le reste de la série (statique) reste l'affaire de Matplotlib et Plotly.

Récapitulatif

- 1. Le rôle.** La visualisation temps réel haute performance : flux de marché, capteurs, rafraîchissement rapide et basse latence.
- 2. La sortie.** Une fenêtre d'application Qt, accélérée GPU — ni image, ni page web.
- 3. Le squelette.** QApplication -> PlotWidget -> app.exec() ; on garde la référence à la courbe renvoyée par plot().
- 4. Les tracés.** plot, ScatterPlotItem, BarGraphItem, ImageItem... créés une fois, jamais recréés.
- 5. Le temps réel.** setData (mettre à jour les données) + QTimer (cadencer sans bloquer l'interface) : le duo clé.
- 6. Compositions.** GraphicsLayoutWidget pour empiler prix/volume, setXLink pour synchroniser les axes.
- 7. Performance.** Fenêtre glissante, axes fixes, OpenGL, une mise à jour par image.
- 8. Le choix.** PyQtGraph pour le temps réel soutenu ; Matplotlib animation pour le simple et l'export GIF/MP4.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Dans quel cas PyQtGraph est-il préférable aux autres bibliothèques de la série ?

Réponse. Pour la visualisation temps réel à haute cadence : un flux de prix en direct, un capteur, un carnet d'ordres rafraîchi des dizaines de fois par seconde. Là où Matplotlib redessine tout et sature, PyQtGraph reste fluide.

Q2. Quelle est la nature de la sortie de PyQtGraph ?

Réponse. Une fenêtre d'application de bureau, bâtie sur Qt et accélérée par le GPU — ni une image figée (Matplotlib), ni une page HTML (Plotly). C'est la version « flux en direct » de l'étape 4 du pipeline.

Q3. Pourquoi garde-t-on la référence à l'objet renvoyé par plot() ?

Réponse. Parce que c'est lui qu'on mettra à jour avec setData(), sans jamais recréer le graphique. Recréer la courbe à chaque image effondrerait la performance ; on ne remplace que ses données.

Q4. Quel est le duo clé de l'animation temps réel, et pourquoi pas une boucle while ?

Réponse. setData (actualiser les données) + QTimer (appeler la mise à jour toutes les N ms). Une boucle while bloquerait la boucle d'événements Qt et figerait la fenêtre ; le QTimer cadence sans bloquer l'interface.

Q5. Cite deux réflexes de performance en temps réel.

Réponse. Garder une fenêtre glissante (ne conserver que les N derniers points au lieu de tout l'historique) et fixer les bornes des axes (setYRange) pour éviter le recalcul d'échelle. On peut aussi activer OpenGL et ne faire qu'une mise à jour par image.

Q6. Quand préférer Matplotlib animation à PyQtGraph ?

Réponse. Pour une animation simple, à basse fréquence, ou qu'on veut exporter en GIF/MP4 : FuncAnimation suffit et réutilise l'API Matplotlib déjà connue. PyQtGraph se justifie quand la cadence est élevée et soutenue, dans une application de bureau.

Document suivant (60) : *Matplotlib 3D (mplot3d) — surfaces, nuages et courbes en trois dimensions : surface de volatilité, relations à trois variables.*